National College of Ireland

Project Submission Sheet

Student Name:	Surya Chandra Raju Kurapati, Saranya Munikannaiah, Nisaraga Holenarasipur Ramesh		
Student ID:	x233969620, x23434821, x24101028		
Programme:	MSc in Data Analytics (MSCDAD_B)	**Year:**	Jan 2025-2026
Module:	Data Intensive and Scalable Systems		
Lecturer:	John Kelly		
Submission Due Date:	28-Jul-2025		
Project Title:	"ConsultIQ-Revisit your visit" – An Intelligent Medical Assistant for recalling clinical visits using Speech and Generative AI		
Word Count:	5169 words		

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature:	Surya Chandra Raju Kurapati, Saranya Munikannaiah, Nisarga Holenarasipur Ramesh
Date:	28-July-2025

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**

Office Use Only	
Signature:	
Date:	

Penalty Applied (if applicable):	

AI Acknowledgement Supplement

Data Intensive and Scalable Systems

Healthcare Insurance Claims Processing: Leveraging Big Data for Policy Renewal Analysis (Descriptive Analytics) and Fraud Detection (predictive Analytics)

Your Name/Student Number	Course	Date
Surya Chandra Raju Kurapati	MSc in Data Analytics	28-Jul-2025
Saranya Munikannaiah	MSc in Data Analytics	28-Jul-2025
Nisarga Holenarasipur Ramesh	MSc in Data Analytics	28-Jul-2025

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool
Notebooklm	Used to understand key concepts of claims fraud detection and examples in detail	https://notebooklm.google.com/

Description of AI Usage

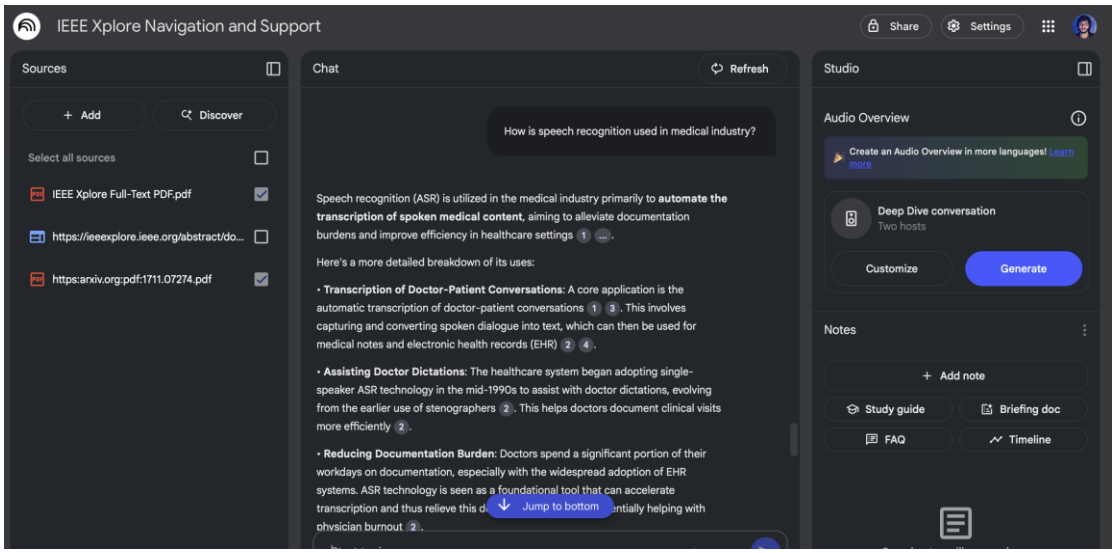
This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
Upon collecting various sources and references relevant to speech recognition and text to vector conversion, used notebooklm to study various content, key objectives of the use case.	
How is speech recognition used in medical industry?	Speech recognition (ASR) is utilized in the medical industry primarily to automate the transcription of spoken medical content, aiming to alleviate documentation burdens and improve efficiency in healthcare settings. Transcription of Doctor-Patient Conversations, Assisting Doctor Dictations,

	Reducing Documentation Burden, Foundation for Further Technologies, Capturing Important Medical Information

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.



Additional Evidence:

[Place evidence here]

Additional Evidence:

[Place evidence here]

"ConsultIQ – Revisit your visit" - An Intelligent Medical Assistant for recalling clinical visits using Speech and Generative AI

Empowering patients and doctors with instant access to clinical visit memories using speech recognition, transformers, and Retrieval-Augmented Generation

1st Surya Chandra Raju Kurapati
Master's in Data Analytics
National College of Ireland
Dublin, Ireland
x23396920@student.ncirl.ie

2nd Saranya Munikannaiah
Master's in Data Analytics
National College of Ireland
Dublin, Ireland
x23434821@student.ncirl.ie

3rd Nisarga Holenarasipur Ramesh
Master's in Data Analytics
National College of Ireland
Dublin, Ireland
x24101028@student.ncirl.ie

***Abstract*—In the healthcare domain, follow-up visits and consultations regarding test results present significant obstacles for both patients and doctors. Particularly when follow-ups are scheduled weeks later, doctors frequently find it difficult to remember the specifics of a patient's prior visit while scanning through scattered notes, lab results, and prescriptions. Similar to this, patients may forget important medical advice or prescribed medications, especially if they are elderly, young, or suffering from cognitive challenges like Alzheimer's. This dependence on memory and fragmented documentation leads to inefficiencies and potential risks in care delivery.**

This paper demonstrates about an AI-powered product "ConsultIQ – Revisit your Visit", designed to bridge the gap by enabling both doctors and patients to instantly revisit past consultations through natural language queries. The system process .wav audio file consisting of clinical conversations between patients and healthcare professionals, converting them into transcriptions using Facebook's wav2vec2-base-960h open-source sentence transformer model. These transcriptions are then structured and indexed for intelligent querying via a Retrieval-Augmented Generation (RAG) framework. The large transcription is first divided into chunks using Langchain, then each chunk is transformed into embeddings using the all-miniLM-6M-v2 embedding model and stored into a FAISS vector database. When a query is received from the user, a similarity search algorithm is applied to retrieve the most relevant chunk, which is then passed as a context to the gemini-2.0-flash large language model to generate accurate and meaningful responses within 5 seconds. Architecturally, the entire pipeline is considered into a three-layered system – API layer, Speech-to-Text layer, and RAG layer.

ConsultIQ reduces the cognitive load for the doctors during the follow-up consultations by providing immediate access to the summarization of prior visits, lab results, and treatment context. This helps in streamlining the patient care. On the other hand, it provides personalized responses to the patients for health related queries by acting as a virtual medical assistant, especially for those who struggle with memory issues and organizing multiple documents.

By integrating Deep Learning, Speech Recognition, Semantic Search, and Generative AI, this paper illustrates a comprehensive and efficient approach to enhancing continuity, accessibility, and intelligence in healthcare interactions.

***Index Terms*—ConsultIQ, Doctor-Patient conversations, Speech-to-Text, Medical summarization, Retrieval Augmented Generation (RAG), FAISS, wav2vec2-base-960h, all-miniLM-6M-v2, embeddings, langchain, API, gemini-2.0-flash, large language model (LLM).**

I. Introduction

In healthcare domain, patients and healthcare professionals face challenges to make sure the information is communicated clearly and continuity of the communication is reached to patient or medical professional. Physicians often has to depend on the fragmented notes, lab reports, and prescriptions gathered over multiple visits, during an emergency situations. This scattered documentation is a problematic situation when time lapses between the visits. Simultaneously, patients particularly the elderly, pediatric populations, or those with cognitive impairments such as Alzheimer's may forget essential medical advice, treatment instructions, or prescribed medications, placing them at risk for complications or non-compliance.

Recent advances in artificial intelligence (AI), speech processing, and retrieval-augmented generation (RAG) models offer promising solutions to this persistent gap. The development of robust speech recognition systems such as wav2vec 2.0 [1], optimized for medical contexts [2], has enabled accurate transcription of doctor–patient dialogues. Furthermore, pretrained embedding models like MiniLM and sentence transformers [5], [6], in conjunction with similarity search engines such as FAISS [7], have paved the way for semantic retrieval from large unstructured datasets. These technologies have seen increasing application in question answering, summarization, and intelligent document retrieval systems, including in healthcare-specific use cases.

To address the limitations in recall and documentation within clinical workflows, this paper introduces ConsultIQ – Revisit your Visit, an AI-powered application designed to augment post-consultation experiences for both patients and healthcare providers. The system processes raw .wav audio recordings of clinical conversations and transcribes them using

Facebook's wav2vec2-base-960h model [1], leveraging deep contextual speech embeddings. These transcriptions are then segmented, embedded using the all-MiniLM-L6-v2 sentence transformer, and indexed using FAISS [7] for rapid similarity-based retrieval.

Queries submitted in natural language are converted into vector representations, enabling retrieval of semantically similar segments from past consultations. These segments are passed to a lightweight generative language model—Gemini-2.0-Flash—for contextual response generation, following the RAG paradigm [8]. This approach allows both patients and doctors to retrieve clinically relevant information within seconds, thus supporting memory recall, personalized healthcare, and operational efficiency.

The system architecture is designed as a three-tiered pipeline encompassing an API layer (built on FastAPI with asynchronous I/O handling), a speech-to-text layer, and a RAG layer. MongoDB is employed for persistent storage of patient metadata and API credentials, while caching strategies and asynchronous processing are used to optimize latency and scalability. Together, these components deliver a responsive, scalable, and user-centric interface for revisiting prior clinical interactions.

By integrating state-of-the-art techniques in deep learning [3], semantic search [5], and speech recognition [2], ConsultIQ contributes to the growing field of interactive medical AI systems, demonstrating how large language models and vector databases can be synergized to improve communication, documentation, and decision support in healthcare settings.

The remainder of this paper is structured as follows: Section II presents Related Work in speech-to-text, medical summarization, and conversational AI. Section III details the Methodology and technological components used. Section IV describes the Tools and Technologies used. Section V and VI explains the Code Implementation and Workflow Execution respectively. Section VII, VIII, IX, discusses about the Evaluation, Results, and Future work. Finally, Section X concludes the paper with insights derived.

II. Related Work

In recent days, the use of AI-based clinical support systems has been incorporating technology in many research areas such as speech recognition, text summarization of medical conversation, semantic retrieval, and the retrieval-augmented generation (RAG). Although the research in all these areas are quite advanced, the current research brings valuable information extraction as well as significant drawbacks when it comes to dynamic, voice-based access to medical visit summaries. This section reviews critical research work and technologies related to our research project.

A. Speech-to-Text in Clinical Contexts

The version of Automatic Speech Recognition (ASR) has significantly changed now with more advanced deep learning models such as Wav2Vec2.0, where raw audio does not need labeled data, but with the self-supervised transformer architecture, it can learn without labeled data [1]. Its resistance to audidfferent audio conditions and possibility to generalize with receive minimal supervisions makes it an ideal candidate in clinical settings where voices, accents and background noise of the patients will further complicate the problem. Nevertheless, unlike in benchmark data such as LibriSpeech, Wav2Vec2.0 might poorly perform on ad-hoc, domain-specific speech such as medical but since it is not trained on clinical speech corpora. Real-life doctor-patient conversation situations have also revealed higher Word Error Rates (WER) than laboratory-tested values [2] casting light on the importance of domain adaptation. Such a limitation notwithstanding, Wav2Vec2 is a powerful baseline architecture to our system, and its design corresponds well with the possibility of fine-tuning or augmentation through medical speech datasets.

B. Medical Text Summarization

Structured summarization of medical information has been used extensively in the medical field as an information for radiology reports and discharge reports with BART, BERT-NAR-BERT (BnB), S2S (Sequence-to-Sequence) [3]. These models are completely producing logical abstractive summaries with a scalable property by training them on large datasets of high-quality texts. However, these models have very minimal utility on conversational transcripts, especially those transcribed by ASR systems, due to their transcription errors, absence of sentence boundaries, and natural language structures. Also, medical summarization is a problem of factual accuracy. Although the reliability of recent LLMs was boosted by prompt-based fine-tuning and reinforcement learning [4], domain specificity remains wanting unless supervised by vast amounts of medical efforts. Therefore, even though LLMs like Gemini Flash 2.0 can do token-based efficient summarization in real time, their text has to be contextualized by retrieval and has to be grounded in real-world medical dialogue. What we present in this work fills this gap by combining the power of LLMs with chunk-level retrieval against clinical conversations.

C. Semantic Embeddings, Vector Retrieval

The retrieval systems of semantics are based on quality embedding to search for critical information parts from the documents. The models, such as Sentence-BERT and the base version all-MiniLM-L6-v2 are good at general-purpose retrieval tasks [5]. MiniLM is efficient, hence can be used by real-time systems such as ConsultIQ as well, where low-latency in embedding and querying is very important. Yet, these models are not specific to a particular domain, and, unless embeddings are trained on biomedical field, a semantic drift may be present even when used on medical terms or abbreviations. The langChain simplifies the search of text in large files by splitting it into smaller chunks. As mentioned, in the Priya, K., et al [6], the Euclidean distance will be calculated between the smaller chunks and the most nearest related chunk will be selected by the search. Although domain-aware models such as BioGPT convey domain-aware vectors,

higher computational costs are frequently accompanied by the inability to perform real-time inference. As such, our decision to work with MiniLM can be characterized as a trade-off between the speed and domain expertise, offsetting the latter inconvenience by vigorous post-retrieval screening with the LLM. The FAISS vector store [7] is a ubiquitous tool in fast search of the vectors and is deployed a great number of times in RAG systems. Its graphic card compatibility and scalability nature makes it suitable inclusion in our architecture.

D. Healthcare Retrieval-Augmented Generation (RAG)

RAG models are a mixture of the strengths involved in information retrieval and text generation. It was proven by Zhang, Ruichen, et al. [8] that RAG is effective at enhancing factual consistency in the open-domain QA setting, as the generation is anchored in external documents. The architectures have been compared and investigated in the healthcare field in the scope of answering the queries over articles and clinical guidelines [9]. However, there are minor changes that have been made to treat RAG with conversational clinical data, which has great potential but extra challenges associated with disfluency, ASR errors, and unstructured data. The novelty of our approach is that we apply RAG to speech with potential evaluation of spoken conversational medical data and generation of semantically informed summaries and answers to questions, without any structuring in speech, which is natural language. It is a move away of document-centric healthcare NLP to be aware of conversation.

E. Identified Gaps and Motivation

To conclude, the existing research only offers great building blocks, such as Wav2Vec2 in speech recognition, transformer-based models in summarization, MiniLM and FAISS in retrieval, but there is no single solution to deal with spoken doctor-patient conversations and summarize them, based on natural-language queries, in real time. Such an absence is exactly what ConsultIQ is targeted at, and these technologies are merged by a single intelligent medical assistant, for a patient and caregiver-centric medical assistant.

III. Methodology

A structured approach based on the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology has been adopted to address the outlined real-world healthcare challenge.

A. Data Collection – Patient-Doctor clinical conversation

For data collection, clinical consultation interaction was simulated and recorded to reflect the real-world doctor-patient interaction. This conversation took place between a person named ABC (Due to compliance reasons, the names are masked), who acted as the patient, and Dr. XYZ, who played the role of the physician. This role-play session was captured in a controlled and conditioned environment using an iPhone at standard voice recording settings and later downloaded as a .wav file. The interaction summary is: “Mr.ABC is here today with concerns about increasing memory problems, including misplacing items, forgetting names, and getting lost while driving. His family also reports he’s been more forgetful than usual. Dr. XYZ will be performing a Mini-mental State Examination and a Montreal Cognitive Assessment. Also doctor has ordered blood tests and, most likely, a brain scan (MRI) to rule out other causes.” From the recorded .wav file, a 16kHz audio waveform was generated to visualize the interaction, illustrating distinct pitch variations between the patient and the doctor. This waveform, as shown in the corresponding Fig. 1, serves as a key input to the speech-to-text pipeline.

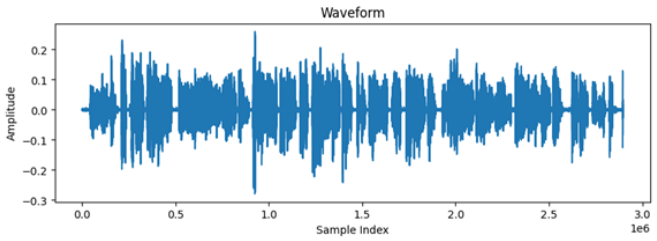


Fig. 1. Audio Waveform

B. Three-Layered Architecture Overview

The core of ConsultIQ is built on a robust three-layer architecture comprising an API layer, a speech-to-text processing layer, and a Retrieval-Augmented Generation (RAG) layer as demonstrated in Fig. 2.

At the entry point, a FastAPI /query endpoint accepts a patient ID along with a natural language query from either a doctor or patient. The system locates the corresponding audio file by appending ”.wav” to the patient ID and searching in local storage. In the Audio-to-text layer, the identified audio file is transcribed using the wav2vec2-base-960h model, a sentence-transformer-based ASR system known for its high performance on English speech [1]. The resulting transcript is then passed to the RAG layer, where it is chunked, embedded using all-MiniLM-6M-v2 model, stored in a FAISS vector index, and queried in real time. Contextually relevant chunks are retrieved and sent to a google gemini-2.0-flash generative large language model to produce accurate, query-specific responses. This architecture converts the natural human spoken language (English) in clinical interactions to intelligent, context-aware, tailored responses to the user query seamlessly.

C. Technical Architecture

The technical architecture of ConsultIQ delivers a full-stack, real-time AI pipeline that transforms spoken clinical interactions into intelligent, contextual responses.

As illustrated in Fig. 3 It begins at the FastAPI /query endpoint, where the user—either a doctor or patient—submits a patient_id and query. This patient_id is used to identify and fetch the respective .wav audio file from the system’s local storage. The audio is processed using librosa built-in Python library to extract and resample the waveform, which is then transformed to text using the facebook/wav2vec2-base-960h

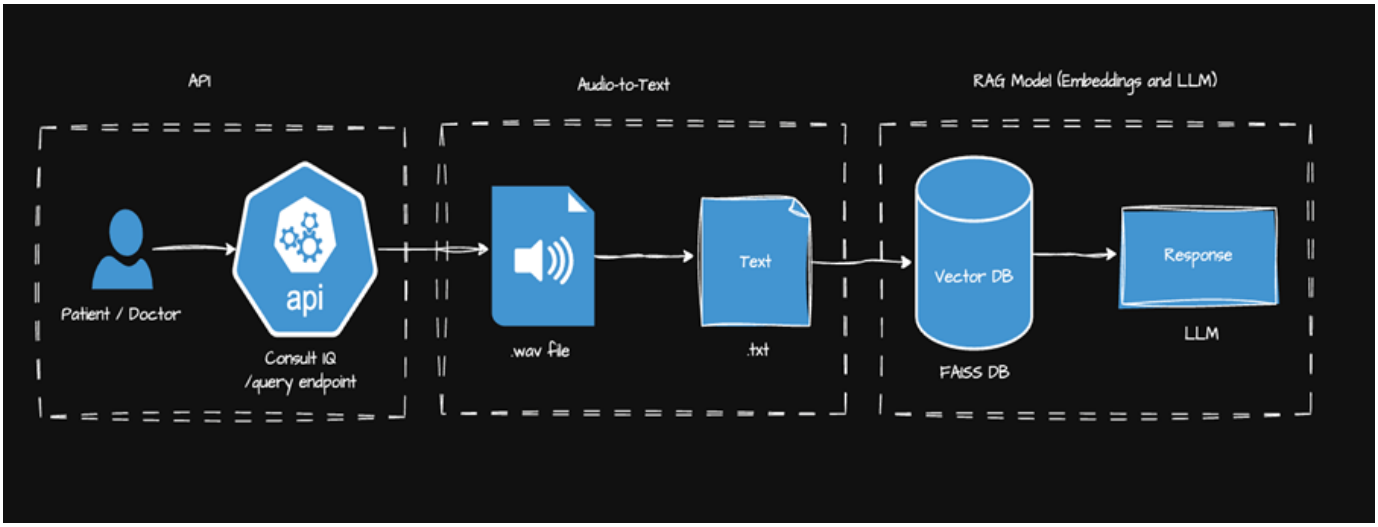


Fig. 2. Three-Layered Architecture Overview

model, which is a robust, self-supervised ASR system well-suited for real-world English speech in clinical environments.

The raw transcript file will be large in real-time. To handle this large file, it is then divided into manageable chunks using Langchain, with each chunk set to approximately 1000 words and a 200-word overlap to preserve context across boundaries. These chunks are embedded into dense semantic vectors using the all-MiniLM sentence transformer, optimized for speed and semantic similarity. The generated embeddings are stored in a FAISS vector database. FAISS database enables rapid retrieval of responses in vector-based storage. This fast retrieval is achieved by performing cosine similarity check between the user query and the embedding stored in FAISS vector databases to calculate the distance between query and chunk, rank relevance, and fetches the top matching chunk's index. Once the index is identified then that top matching chunk is passed along with the user query to gemini-2.0-flash to generate an informed and grounded response. The LLM model gemini-2.0-flash is capable of performing this task in less than 5 seconds in real-time, depending on the input size.

To enhance efficiency, MongoDB is integrated as a lightweight metadata store, which is primarily used to cache transcription results and store the Gemini API key securely, removing the need for hard-coded credentials. This allows the system to reduce the computational redundancy by reusing the transcripts for any other query made user for the same patient. Along with this, to ensure the scalability and non-blocking processing of API calls, asyncio is used. This three-layered architecture helps to achieve a low-latency, robust outcome by converting the voice input to intelligent clinical insights.

D. Components

This section explains each core component of the system that collectively power the end-to-end workflow, from raw audio input to intelligent, context-aware medical responses.

"/query" endpoint:

The /query endpoint, built using FastAPI, serves as the primary entry point for users including doctors and patients, to interact with the system. It accepts two inputs: patient_id and a natural language query as illustrated in Fig. 4.

Internally, the system appends the .wav extension to the patient_id and searches for the corresponding audio file in local storage (e.g., patient123.wav). This design ensures a direct mapping between the user and their recorded conversation. The endpoint is asynchronous, leveraging Python's asyncio for non-blocking I/O calls, which enables efficient handling of concurrent requests and real-time responsiveness.

Speech-to-Text workflow: The .wav audio file is passed through a detailed speech-to-text pipeline leveraging Wav2Vec2. As illustrated in Fig. 5. Let's consider a sample audio that is 3 seconds long, recorded at 16,000 samples per second (16kHz). Using librosa, this is converted into a waveform, a 1D PyTorch tensor of shape [1, 48000] (3 seconds × 16000 Hz). This waveform is split into frames using a 20ms stride, resulting in 3000ms / 20ms = 150 frames. These frames are processed through Wav2Vec2's 7-layer convolutional encoder, which extracts acoustic features. The encoder maps these frames to a latent space, followed by projection to a vocabulary space using softmax to output logits. The argmax of these logits gives the most likely token at each timestep, forming a sequence of predicted tokens [1, 150]. Finally, the CTC (Connectionist Temporal Classification) decoder translates these into English words [10].

Transcription Chunks: The generated transcript can be extremely lengthy, often exceeding thousands of tokens, too large to pass directly into a language model like Gemini due to token limit constraints and potential rate limit errors. To overcome this, the full transcript is broken down using LangChain's CharacterTextSplitter into chunks of 1000 words, with an overlap of 200 words to maintain context continuity

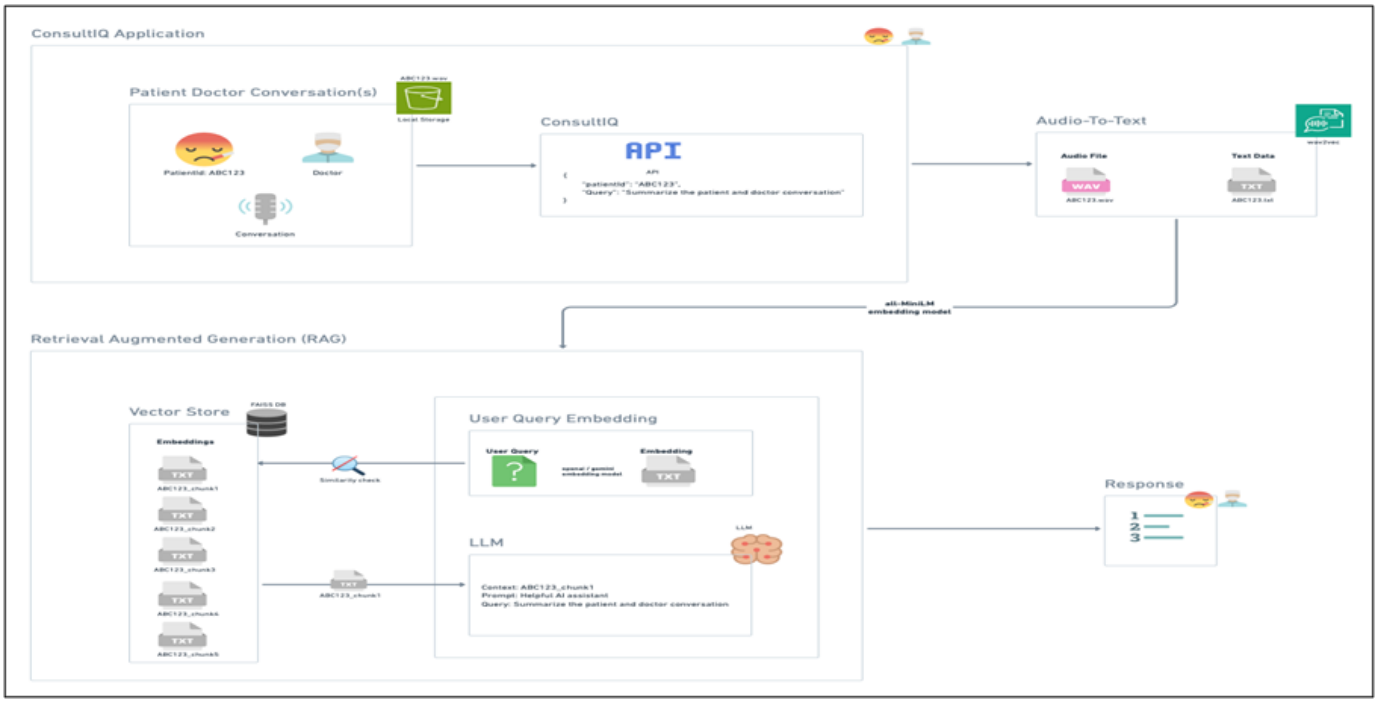


Fig. 3. Technical Architecture

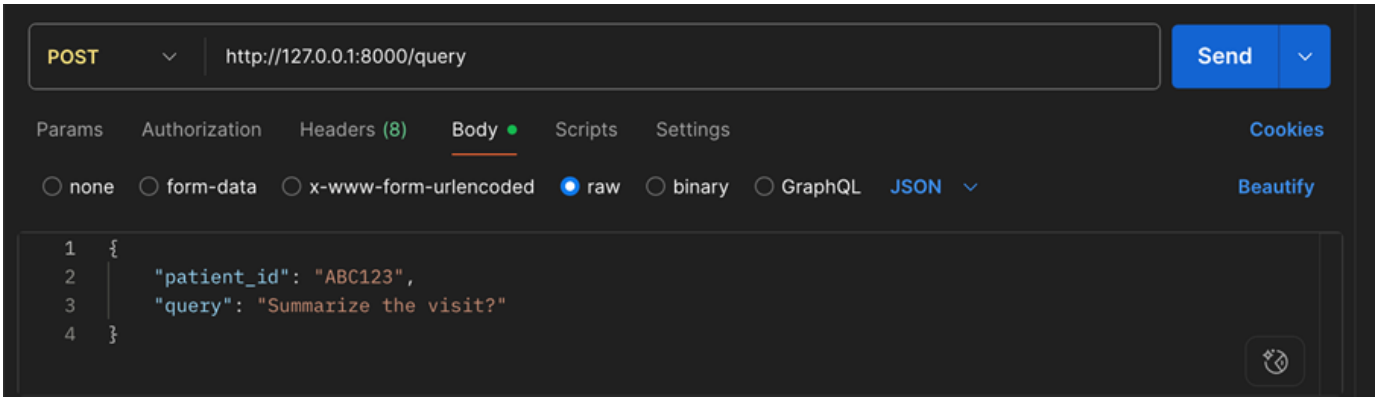


Fig. 4. API Endpoint

across segments. This intelligent chunking ensures that relevant parts of the conversation are not lost during retrieval and LLM interaction, while also keeping within model input limits. A simple Python class wraps the LangChain splitter to produce these consistent and reusable chunks from any text input.

Embedding: Each chunk is converted into a high-dimensional vector using the all-MiniLM-L6-v2 embedding model. This model transforms input text into dense, semantically meaningful vectors of dimension 384. It is a crystal-clear transformer that ensures high performance with a compact and lightweight architecture, making it suitable for fast and real-time applications. These embeddings capture nuanced meanings of clinical conversations, enabling accurate similarity comparison against user queries.

Vector Store (FAISS) and Similarity Search: To enable semantic search, the 384-dimensional embeddings generated from each transcript chunk are stored in a FAISS (Facebook AI Similarity Search) vector index, which supports fast and scalable nearest-neighbour retrieval. The embeddings, along with their corresponding document chunks, are indexed using faiss.IndexFlatL2 [11], which performs similarity calculations based on L2 distance or cosine similarity. These components are saved in a binary .bin file, while the original chunks are serialized in a .pkl file for efficient access. When a user query is received, it is embedded using the same all-MiniLM-L6-v2 model to maintain consistency. The query embedding is then matched against the stored vectors using index.search to retrieve the top-k most similar chunks. These chunks, which hold the most relevant transcript segments, are passed as con-

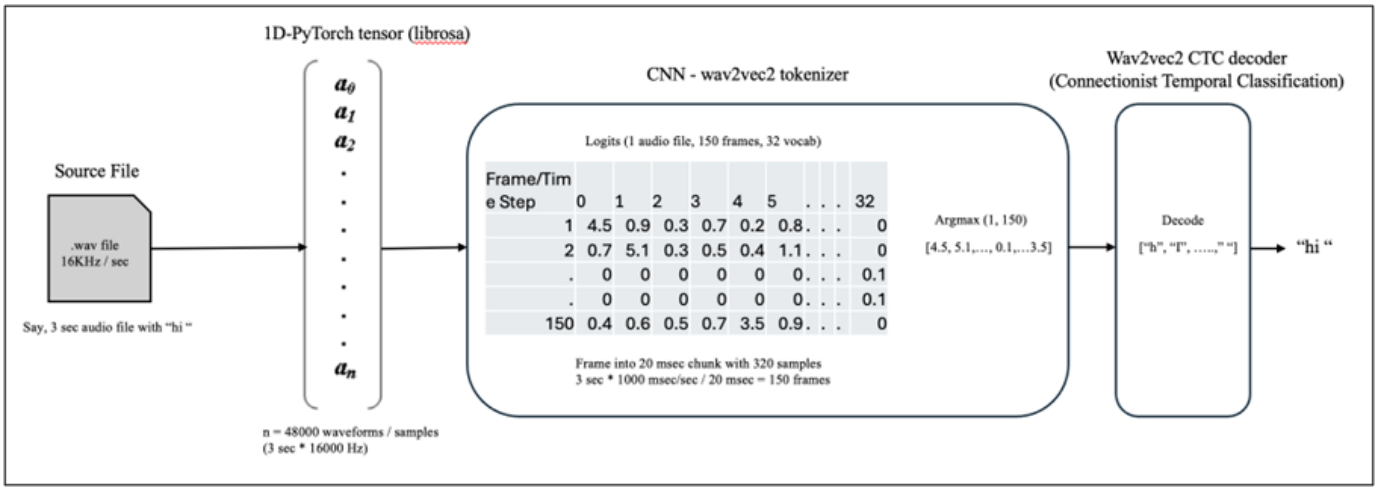


Fig. 5. Speech-to-Text workflow

textual input to the language model. The entire indexing and retrieval process is encapsulated within a dedicated Retriever class, which supports index building, loading, and searching-ensuring a modular, reusable, and production-ready search pipeline.

LLM (Large Language Model): The final stage of the pipeline utilizes Google gemini-2.0-flash, a lightweight and fast large language model optimized for real-time inference. It takes the user's query and the context chunk retrieved from FAISS to generate an accurate, factually grounded answer. Gemini Flash balances generation speed with high-quality outputs and performs well even under limited context windows, making it ideal for this use case.

IV. Code Implementation

The technical code artifact, illustrated in Fig. 6, is implemented in a modular and object-oriented fashion using Python, with the FastAPI framework serving as the core web interface. The main application logic resides in app/main.py, where a RESTful POST endpoint /query accepts a JSON payload containing a patient_id and query. Upon receiving a request, the system locates the corresponding audio file from the /data/ directory using a helper function and verifies its existence. The audio is then processed through the AudioProcessor class, which utilizes Facebook's wav2vec2-base-960h model to transcribe the raw .wav waveform into human-readable text. Internally, this step converts audio into float-based waveforms using librosa, tokenizes the waveforms, passes them through the wav2vec2 encoder, and finally decodes the logits into words using a CTC decoder. The transcribed text is passed to the TextProcessor, which segments the transcript into overlapping chunks using Langchain's CharacterTextSplitter, enhancing semantic coherence and preserving context. These chunks are then embedded using the Embedder module, which leverages the all-MiniLM-L6-v2 model from the sentence-transformerslibrary to convert each chunk into a 384-dimensional dense vector. The resulting vectors and their corresponding text chunks are used to either construct or load a FAISS-based similarity index within the Retriever class. This vector index enables efficient top-k retrieval of semantically similar text chunks based on a user query, which is also embedded using the same model for consistency.

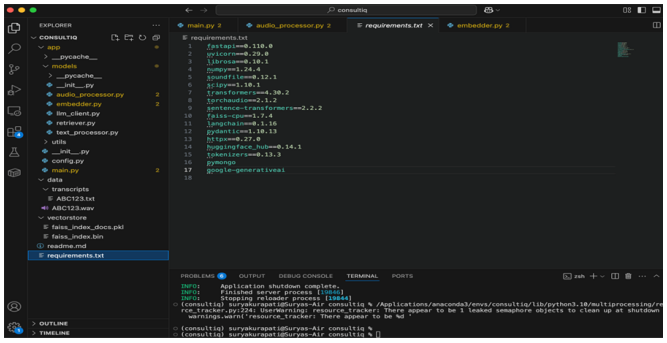


Fig. 6. Code Setup

The top-ranked chunk is subsequently combined with the user's query and passed to the LLMClient, which constructs a prompt tailored for Google's gemini-2.0-flash model. The model then generates a context-aware response based on the retrieved information and the user's query. The system is fully asynchronous, utilizing FastAPI's non-blocking async/await model to ensure responsiveness during I/O-heavy tasks such as audio reading, model inference, and embedding. MongoDB integration (described elsewhere) is used for securely storing configuration metadata like API keys or user data.

V. Tools and Technology

Table I illustrates the tools and technologies selected and leveraged across various stages of the workflow. IT also details the purpose of choosing each component based on their specialization such as seamless audio processing, semantic retrieval, and intelligent response generation.

TABLE I
TOOLS AND TECHNOLOGIES USED IN THE CONSULTIQ SYSTEM

Category	Tool / Library	Usage / Purpose
Audio Processing	`librosa`	Load `.wav` files, convert to waveform, resample, and extract audio features.
Speech Recognition	`facebook/wav2vec2-base-960h`	Pretrained deep learning model for English speech-to-text conversion.
Text Splitter	`langchain`	Splits large text files into chunks with overlapping words for better context retention.
Embedding	`all-MiniLM`	Sentence transformer that converts text chunks into dense vector embeddings.
Vector Database	`FAISS`	Stores and retrieves similar vectors using approximate nearest-neighbor search.
Large Language Model	`gemini-2.0-flash`	Generates natural language answers to questions based on retrieved context.
API Server	`FastAPI`	High-performance asynchronous Python API framework for serving backend endpoints.
Database	`MongoDB`	Stores metadata such as API keys, logs, and session information.
Async Processing	`asyncio`	Manages asynchronous I/O operations to support non-blocking request handling.
Data Processing	`NumPy`	Performs mathematical operations and waveform array conversions.

VI. WORKFLOW EXECUTION

The entire workflow can be set up and executed by following the instructions provided. As a first step, all the necessary dependencies should be installed in the system environment where the workflow must be executed. The installation can be done using the command pip install -r requirements.txt. Once the environment is ready, the doctor-patient conversation, .wav audio file named with patient's identifier (e.g., ABC123.wav) is placed in the /data/ directory. The system is then launched using the FastAPI server command uvicorn app.main:app –reload, which initializes core components including the wav2vec2 speech recognition model, the sentence embedding model, and the FAISS similarity index.

Upon successful server startup, the application is ready to process queries via a POST request sent to http://127.0.0.1:8000/query. The payload must be a JSON object containing two fields: patient_id and query. This allows the system to retrieve the associated audio file, generate its transcription, segment and embed the text, and perform vector-based retrieval.

An example query invocation is shown below: curl -X POST ”http://127.0.0.1:8000/query” -H ”Content-Type: application/json” -d ’”patient_id”: ”ABC123”, ”query”: ”Summarize the visit?”’

VII. RESULTS AND OUTCOME

Fig. 7 shows a successful POST request made to the http://127.0.0.1:8000/query endpoint of the ConsultIQ system via Postman. The request body contains a patient_id (”ABC123”) and a user query (”Summarize the visit?”). Upon processing the request, the system retrieves the associated .wav file, transcribes it using the wav2vec2 model, segments the text, embeds the chunks, performs similarity search via FAISS, and generates a context-aware response using a Gemini-2.0-Flash-backed LLM. The returned JSON response summarizes the patient visit in 1.32 seconds, noting memory-related concerns reported by the patient and his wife, and outlines the planned clinical assessments, including cognitive tests and a possible MRI - demonstrating the end-to-end functionality of the retrieval-augmented pipeline.

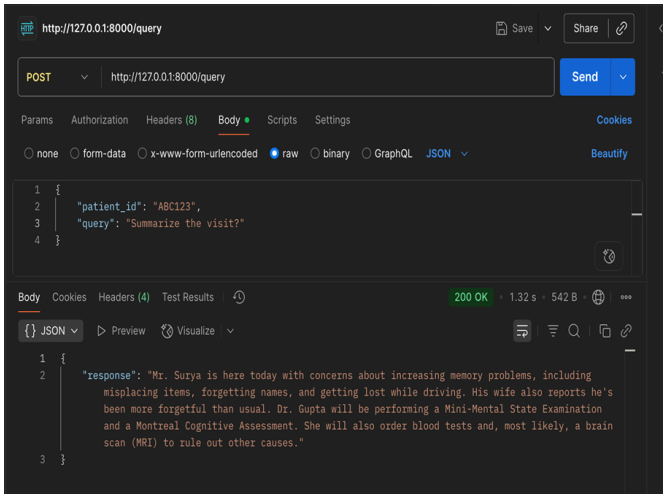


Fig. 7. Response

VIII. EVALUATION

The evaluation of ConsultIQ's response quality was conducted using both manual assessment and automated ROUGE metrics. For the manual evaluation, ground truth summaries were considered by listening to the original patient audio recordings and creating concise summaries of the discussed symptoms and medical history. The LLM-generated responses were then compared against these reference summaries for semantic coherence, factual accuracy, and clinical relevance. This qualitative method allowed an assessment of whether the system retained the correct intent and meaning, even when the phrasing or terminology differed slightly from the gold standard.

Complementing the manual evaluation, automated scoring was performed using the ROUGE (Recall-Oriented Understudy for Gisting Evaluation) metric, which measures lexical overlap between generated responses and reference summaries. Specifically, the evaluate library was used to load and compute the ROUGE-1, ROUGE-2, ROUGE-L, and ROUGE-Lsum scores. Table II illustrates the scores. These scores indicate a moderate degree of content overlap, which is in line with the performance of many baseline summarization models, particularly in challenging domains such as conversational

medical transcripts.

TABLE II
ROUGE EVALUATION METRICS FOR SUMMARIZATION QUALITY

Metric	Score
ROUGE-1	0.3507
ROUGE-2	0.1504
ROUGE-L	0.2910
ROUGE-Lsum	0.2985

IX. CONCLUSION

ConsultIQ effectively addresses the memory and documentation challenges commonly encountered by both healthcare professionals and patients during follow-up consultations. By transforming raw audio recordings of clinical interactions into structured, searchable data using wav2vec2-based speech recognition and Retrieval-Augmented Generation, the system enables precise and timely access to prior medical information. Doctors no longer need to sift through fragmented notes or manually search for past prescriptions and lab reports, as ConsultIQ's embedding and vector retrieval mechanism surfaces the most relevant conversational context in real time. Likewise, patients—especially those with cognitive limitations—can revisit medical advice or treatment plans through natural language queries, reducing their dependency on memory. The integration of FAISS indexing, LangChain chunking, and Gemini-based LLM responses ensures that information retrieval is both semantically accurate and context-aware. By bridging these information gaps, ConsultIQ streamlines clinical workflows, enhances decision-making, and promotes safer, more personalized care delivery across patient populations.

X. FUTURE WORK

Future enhancements of the ConsultIQ system can significantly benefit from upgrading the transcription pipeline, embedding mechanisms, and overall infrastructure. While the current system uses facebook/wav2vec2-base-960h for speech-to-text, it can be enhanced by migrating to OpenAI's Whisper model, which offers improved transcription accuracy across diverse accents, speaker styles, and noisy backgrounds. Whisper's multilingual capabilities also open doors for cross-linguistic clinical applications, making it a versatile upgrade. Similarly, the current embedding module based on all-MiniLM-L6-v2 could be replaced or augmented with more context-aware models such as OpenAI's text-embedding-3-large, which capture deeper semantic meaning. These changes would improve both retrieval accuracy and downstream LLM performance.

The system can also transition to a cloud-native architecture for better scalability and reliability. Audio files currently stored locally in the /data directory may be migrated to AWS S3, providing secure, redundant storage and seamless access across distributed environments. For fast in-memory access and latency reduction, Redis can be introduced as a caching layer to store frequently accessed transcripts or vector embeddings. While FAISS is effective for local vector similarity search, it can be substituted with more scalable, managed vector databases such as Pinecone. Importantly, the Google Gemini-2.0-Flash model used for response generation remains performant and can continue to serve the system reliably. Lastly, the current FastAPI and MongoDB stack can be containerized and deployed using services like AWS ECS or Google Cloud Run, while asyncio continues to power asynchronous workflows such as real-time inference and multi-query batching.

ACKNOWLEDGMENT

We would like to express our sincere gratitude to **Mr.John Kelly** for his invaluable insights, constructive feedback, and continuous support throughout the course of this study.

We are also grateful to each other — Surya Chandra Raju Kurapati, Saranya Munikannaiah and Nisarga Holenarasipur Ramesh — for the collaborative effort, shared learning, and mutual encouragement that made this product design and implementation possible.

REFERENCES

[1] Y. Lim, N. Kim, S. Yun, S. Kim, and S. I. Lee, "A preliminary study on wav2vec 2.0 embeddings for text-to-speech," in *Proc. 2021 Int. Conf. Inf. Commun. Technol. Converg. (ICTC)*, Jeju, South Korea, Oct. 2021, pp. 343–347.

[2] D. Jaunzeikare *et al.*, "Speech recognition for medical conversations," *arXiv preprint arXiv:1711.07274*, 2017.

[3] M. G. Sohrab, M. Asada, M. Rikters, and M. Miwa, "BERT-NAR-BERT: A non-autoregressive pre-trained sequence-to-sequence model leveraging BERT checkpoints," *IEEE Access*, vol. 12, pp. 23–33, 2023.

[4] J. Duan, F. Lu, and J. Liu, "MVP: Optimizing multi-view prompts for medical dialogue summarization," in *Proc. 2023 IEEE Int. Conf. Bioinf. Biomed. (BIBM)*, Istanbul, Turkey, Dec. 2023, pp. 500–507.

[5] H. Palangi *et al.*, "Deep sentence embedding using long short-term memory networks: Analysis and application to information retrieval," *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 24, no. 4, pp. 694–707, Apr. 2016.

[6] K. Priya *et al.*, "Enhancing Q&A systems with multilingual text conversion and speech integration: Harnessing the power of LangChain and large language models," in *Proc. 2024 Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solut. (CSITSS)*, Bangalore, India, Nov. 2024, pp. 1–6.

[7] J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Sep. 2019.

[8] R. Zhang *et al.*, "Interactive AI with retrieval-augmented generation for next generation networking," *IEEE Netw.*, vol. 38, no. 6, pp. 414–424, 2024.

[9] Y. Kim, J. Wu, Y. Abdulle, and H. Wu, "MedExQA: Medical question answering benchmark with multiple explanations," *arXiv preprint arXiv:2406.06331*, 2024.

[10] A. Das, J. Li, R. Zhao, and Y. Gong, "Advancing connectionist temporal classification with attention modeling," in *Proc. 2018 IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, Calgary, AB, Canada, Apr. 2018, pp. 4769–4773.

[11] P. N. Singh, S. Talasila, and S. V. Banakar, "Analyzing embedding models for embedding vectors in vector databases," in *Proc. 2023 IEEE Int. Conf. ICT in Business Industry & Government (ICTBIG)*, Hyderabad, India, Dec. 2023, pp. 1–7.